

Curso Profesional de Entity Framework Core 8

Módulo 3 - Consultas con LINQ

PRÁCTICAS

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 3.1 — Fundamentos de LINQ to Entities	3
Punto 3.2 — Consultas básicas: Where, OrderBy y ThenBy	8
Punto 3.3 — Proyecciones con Select y tipos anónimos	14
Punto 3.4 — Proyecciones a DTOs	19
Punto 3.5 — Consultas de agregación: Count, Sum, Average, Min y Max	24
Punto 3.6 — Agrupaciones con proyección	30
Punto 3.7 — Joins y navegación en consultas	35
Punto 3.8 — Eager Loading con Include y ThenInclude	41
Punto 3.9 — Lazy Loading: configuración, funcionamiento y riesgos	47
Punto 3.10 — Explicit Loading	52
Punto 3.11 — Composición de consultas y ejecución diferida	57
Punto 3.12 — Buenas prácticas en el acceso a datos y composición de consultas	62

Curso Profesional de Entity Framework Core 8

Módulo 3 — Prácticas: Consultas con LINQ

Autor: JAIME GALLO

Cada práctica trabaja sobre un checkpoint completo en M03/PROYECTO/3.x. La progresión es acumulativa: 3.1 parte de M2.12 y cada carpeta posterior conserva el estado anterior más el concepto nuevo.

Punto 3.1 — Fundamentos de LINQ to Entities

Contexto del proyecto

Este checkpoint continúa M02/PROYECTO/2.12. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Fundamentos de LINQ to Entities** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.1
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `Consulta`, `ObtenerSqlFundamentos`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ConsultasLinqUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ConsultasLinqUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");
    }
}
```

CSHARP

```

var enMemoria = _unidad.Ordenes.ObtenerTodas().Where(o => o.Cliente == "Constructora del Norte").ToList();
var consulta = _unidad.Ordenes.Consulta().Where(o => o.Cliente == "Constructora del Norte").OrderBy(o =>
o.FechaCreacion);
Console.WriteLine("Consulta IQueryable construida: aún no se ha materializado.");
var enSql = consulta.ToList();
if (enMemoria.Count != 3 || enSql.Count != 3) throw new InvalidOperationException("Comparación IEnumerable/IQueryable inesperada.");
Console.WriteLine($"Enumerable: {enMemoria.Count} | IQueryable: {enSql.Count}");
Console.WriteLine(_unidad.Ordenes.ObtenerSqlFundamentos());
}
}

```

Explicación línea a línea del caso de uso 3.1

Línea 1	using AceriaData.Application.Interfaces; → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.
Línea 3	namespace AceriaData.Application.UseCases; → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	public sealed class ConsultasLinqUseCase → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.
Línea 8	public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	public void Ejecutar() → Define la operación docente que ejecutará el composition root.
Línea 11	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ==="); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	var enMemoria = _unidad.Ordenes.ObtenerTodas().Where(o => o.Cliente == "Constructora del Norte").ToList(); → Materializa primero todas las órdenes y aplica después el filtro en memoria, creando deliberadamente el contraste con la consulta IQueryable.
Línea 14	var consulta = _unidad.Ordenes.Consulta().Where(o => o.Cliente == "Constructora del Norte").OrderBy(o => o.FechaCreacion); → Compone filtro y ordenación sobre IQueryable sin materializar; EF Core conserva el árbol de expresión hasta la ejecución.
Línea 15	Console.WriteLine("Consulta IQueryable construida: aún no se ha materializado."); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 16	var enSql = consulta.ToList(); → Materializa la secuencia en una lista; si el origen sigue siendo IQueryable, aquí se ejecuta SQL.
Línea 17	if (enMemoria.Count != 3 enSql.Count != 3) throw new InvalidOperationException("Comparación IEnumerable/IQueryable inesperada."); → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	Console.WriteLine(\$"Enumerable: {enMemoria.Count} IQueryable: {enSql.Count}"); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 19	Console.WriteLine(_unidad.Ordenes.ObtenerSqlFundamentos()); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20	} → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 21	} → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ConsultasLinqUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.1 OK");
```

CSHARP

Explicación línea a línea de Program.cs 3.1

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Inicia la construcción de la configuración de la aplicación.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Fija como base el directorio de ejecución para localizar <code>appsettings.json</code> .
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer <code>ConnectionStrings:AceriaDB</code> .
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Obtiene la cadena <code>AceriaDB</code> desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión <code>AceriaDB</code> , evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del <code>composition root</code> .
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra <code>DbContext</code> , repositorio y unidad de trabajo de <code>Infrastructure</code> .
Línea 20	<code>services.AddScoped<ConsultasLinqUseCase>();</code> → Registra el caso de uso con ciclo de vida <code>Scoped</code> para compartir el mismo ámbito que <code>DbContext</code> .
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios <code>IDisposable</code> al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio <code>Scoped</code> consumido incorrectamente desde un <code>Singleton</code> .
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito <code>Scoped</code> que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.1 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

3.1 OK

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Materializar antes de filtrar mueve el trabajo a memoria y oculta el SQL que se quería estudiar.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.1

Diagnóstico técnico

El objetivo no es sólo obtener el mismo número de órdenes con `IEnumerable<T>` e `IQueryable<T>`, sino comprobar dónde se ejecuta el filtro. Si se materializa primero, el SQL trae todas las filas visibles y el predicado se aplica en memoria; si se mantiene `IQueryable<T>`, el predicado llega al `WHERE` de SQL Server.

Reto resuelto y verificación adicional

Construye una consulta de órdenes pendientes del Norte, ordenadas de más reciente a más antigua y proyectadas a número y cliente. El pipeline correcto es `Where` → `OrderByDescending` → `Select` → `ToList`. Antes de materializar, usa `ToQueryString()` y verifica `WHERE`, `ORDER BY` y una proyección mínima.

Errores comunes revisados

Error	Causa	Solución
Filtro se aplica en memoria	Se llamó a <code>ToList</code> antes del filtro	Aplicar <code>ToList</code> al final
Expresión no traducida en filtro u operación de servidor	Se usó lógica que el proveedor no puede traducir fuera de la proyección superior	Reescribir con expresiones traducibles o cambiar explícitamente a evaluación cliente cuando esté justificado

Error	Causa	Solución
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
ToQueryString ejecuta la consulta	No, solo la construye	Verificar que no se llama a ToList
IQueryable expuesto en el repositorio	Anti-patrón	Encapsular en métodos específicos (Módulo 4)
Falta el using de EF Core	El método ToQueryString no está disponible	Añadir using Microsoft.EntityFrameworkCore;

Analogía operativa

LINQ to Entities en una acería es como el lenguaje que usa el jefe de planta para pedir informes al archivo central. El jefe no va al archivo a buscar los documentos: envía una orden con las condiciones (cliente, estado, fecha) y el archivo devuelve solo los documentos que cumplen esas condiciones. Si el jefe pidiera todos los documentos y luego los filtrara él mismo, perdería tiempo y esfuerzo. Eso es lo que ocurre en este ejemplo cuando primero se llama a `ToList()` y después se filtra como `IEnumerable`: los datos ya están materializados y el filtro posterior se ejecuta en el cliente. `IEnumerable` por sí solo no significa que toda secuencia esté previamente cargada en memoria. La ejecución diferida es como preparar la orden de búsqueda antes de enviarla: el jefe puede añadir condiciones hasta que esté seguro, y solo entonces la envía. La materialización es el momento en que se envía la orden y se recibe la respuesta. `ToQueryString()` es como pedir una copia de la orden sin enviarla: permite revisar el SQL que EF Core ha construido sin ejecutar esa consulta.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Añade un segundo filtro opcional por estado sin materializar hasta el final y compara el SQL.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.1 OK, el siguiente estado parte exactamente de esta solución y añade **Consultas básicas: Where, OrderBy y ThenBy**.

Punto 3.2 — Consultas básicas: Where, OrderBy y ThenBy

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.1. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Consultas básicas: Where, OrderBy y ThenBy** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.2
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerPendientesPorCliente`, `ObtenerPorEstadoOrdenadasPorFecha`, `ObtenerPorRangoDeFechas`, `ObtenerPorClienteOrdenadas`, `ObtenerPorClienteYRangoDeFechas` y `ObtenerSqlConsultaBasica`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ConsultasBasicasUseCase.cs`

CSHARP

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ConsultasBasicasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== WHERE, ORDERBY Y THENBY ===");
        var norte = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");
        var pendientes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");
        var rango = _unidad.Ordenes.ObtenerPorRangoDeFechas(new DateTime(2024,1,1), new DateTime(2024,12,31));
        var ordenadas = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");
        var combinada = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", new DateTime(2024,1,1), new DateTime(2024,12,31));
        if (norte.Count != 2 || pendientes.Count != 3 || rango.Count != 5 || ordenadas.Count != 3 || combinada.Count != 3) throw new InvalidOperationException("Resultados de filtros/ordenaciones inesperados.");
        if (!ordenadas.Select(o => o.NumeroOrden).SequenceEqual(new[] { "OF-2024-0004", "OF-2024-0003", "OF-2024-0001" })) throw new InvalidOperationException("ThenByDescending no produjo el orden esperado.");

        Console.WriteLine($"Norte pendientes: {norte.Count} | Pendientes: {pendientes.Count} | Rango: {rango.Count} | Norte ordenadas: {ordenadas.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlConsultaBasica());
    }
}
```

Explicación línea a línea del caso de uso 3.2

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application, en especial <code>IUnidadDeTrabajo</code> e <code>IOrdenRepositorio</code> , que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class ConsultasBasicasUseCase</code> → Declara el caso de uso que valida filtros y ordenaciones del punto 3.2.
Línea 6	<code>{</code> → Abre el bloque de la clase.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de <code>DbContext</code> .
Línea 8	<code>public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación que ejecuta todas las comprobaciones E2E de este checkpoint.
Línea 11	<code>{</code> → Abre el cuerpo del método <code>Ejecutar</code> .
Línea 12	<code>Console.WriteLine("=== WHERE, ORDERBY Y THENBY ===");</code> → Identifica en consola la demostración que se está ejecutando.
Línea 13	<code>var norte = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");</code> → Ejecuta el filtro compuesto por cliente y estado y conserva las órdenes pendientes del Norte.

Línea 14	<code>var pendientes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");</code> → Obtiene las órdenes pendientes ordenadas por fecha descendente para validar <code>OrderByDescending</code> .
Línea 15	<code>var rango = _unidad.Ordenes.ObtenerPorRangoDeFechas(new DateTime(2024,1,1), new DateTime(2024,12,31));</code> → Filtra por intervalo de fechas y valida la composición de comparaciones sobre <code>DateTime</code> .
Línea 16	<code>var ordenadas = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");</code> → Ejecuta la consulta que encadena <code>OrderBy</code> y <code>ThenByDescending</code> para el cliente indicado.
Línea 17	<code>var combinada = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", new DateTime(2024,1,1), new DateTime(2024,12,31));</code> → Combina cliente, rango de fechas y ordenación múltiple en una consulta encapsulada.
Línea 18	<code>if (norte.Count != 2 pendientes.Count != 3 rango.Count != 5 ordenadas.Count != 3 combinada.Count != 3) throw new InvalidOperationException("Resultados de filtros/ordenaciones inesperados.");</code> → Comprueba los cardinales deterministas de las cinco consultas y hace fallar el E2E ante cualquier regresión.
Línea 19	<code>if (!ordenadas.Select(o => o.NumeroOrden).SequenceEqual(new[] { "OF-2024-0004", "OF-2024-0003", "OF-2024-0001" })) throw new InvalidOperationException("ThenByDescending no produjo el orden esperado.");</code> → Verifica el orden exacto de las tres órdenes del Norte, por lo que no basta con obtener las filas correctas: la ordenación también debe ser correcta.
Línea 20	<code>Console.WriteLine(\$"Norte pendientes: {norte.Count} Pendientes: {pendientes.Count} Rango: {rango.Count} Norte ordenadas: {ordenadas.Count}");</code> → Resume los cardinales validados para que la ejecución sea inspeccionable.
Línea 21	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlConsultaBasica());</code> → Imprime el SQL generado para la consulta de referencia con filtro y ordenación múltiple.
Línea 22	<code>}</code> → Cierra el cuerpo del método <code>Ejecutar</code> .
Línea 23	<code>}</code> → Cierra la clase <code>ConsultasBasicasUseCase</code> .

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ConsultasBasicasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

```

context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.2 OK");

```

Explicación línea a línea de Program.cs 3.2

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.
Línea 16	?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	var services = new ServiceCollection(); → Crea el contenedor de servicios del composition root.
Línea 19	services.AddAceriaInfrastructure(connectionString); → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	services.AddScoped<ConsultasBasicasUseCase>(); → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	using var provider = services.BuildServiceProvider(new ServiceProviderOptions → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.

Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.2 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infraestructura o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.2 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un segundo `OrderBy` sustituye la ordenación anterior; para añadir criterios se usa `ThenBy`.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.2

Diagnóstico técnico

Prueba filtros por cliente, estado y fechas antes de combinarlos. `ThenBy` añade un criterio a un `OrderBy`; un segundo `OrderBy` sustituiría el criterio anterior. Los métodos de cadena traducibles no deben confundirse con formas necesariamente óptimas para índices.

Reto resuelto y verificación adicional

Para Constructora del Norte durante 2024, valida el método acumulativo que filtra por cliente/rango y ordena por estado ascendente y fecha descendente. El SQL debe parametrizar las fechas y no concatenar valores.

Errores comunes revisados

Error	Causa	Solución
Filtro se aplica en memoria	Se llamó a <code>ToList</code> antes del filtro	Aplicar <code>ToList</code> al final
Ordenación no aplicada	Se llamó a <code>OrderBy</code> después de <code>ToList</code>	Aplicar <code>OrderBy</code> antes de <code>ToList</code>
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
<code>IQueryable</code> expuesto en el repositorio	Anti-patrón	Encapsular en métodos específicos (Módulo 4)
Filtro por fecha incorrecto	Se usó <code>==</code> en lugar de <code>>=</code> y <code><=</code>	Usar operadores de comparación
<code>ThenBy</code> sin <code>OrderBy</code>	Se llamó a <code>ThenBy</code> sin <code>OrderBy</code> previo	Llamar a <code>OrderBy</code> primero
Falta el using de EF Core	El método <code>ToQueryString</code> no está disponible	Añadir using <code>Microsoft.EntityFrameworkCore</code> ;

Analogía operativa

Las consultas básicas con LINQ en una acería son como las órdenes de búsqueda que el jefe de planta envía al archivo central. Where es el filtro: "solo quiero las órdenes del cliente Constructora del Norte". `OrderBy` es el criterio de ordenación: "ordénalas por fecha de creación". `ThenBy` es el criterio secundario: "y dentro de cada fecha, ordénalas por estado". El archivo central recibe la orden, busca en sus índices y devuelve exactamente lo que se pidió. Si el jefe pidiera todos los documentos y los ordenara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se materializa antes de filtrar. Las consultas encapsuladas en el repositorio son como las ventanillas especializadas del archivo: cada una sabe qué buscar y cómo ordenarlo. El jefe no necesita conocer el archivo por dentro: solo pide lo que necesita a la ventanilla correspondiente. Así funcionan las consultas básicas con LINQ: se filtran y se ordenan en el servidor, y los resultados se devuelven ya preparados.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Crea una consulta de un cliente en un rango de fechas ordenada por estado y fecha descendente.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.2 OK, el siguiente estado parte exactamente de esta solución y añade **Proyecciones con `Select` y tipos anónimos**.

Punto 3.3 — Proyecciones con Select y tipos anónimos

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.2. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Proyecciones con Select y tipos anónimos** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.3
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerClientesUnicos`, `ObtenerResumenes`, `ObtenerResumenesPorEstado`, `ObtenerOrdenesConTotales`, `ObtenerSqlProyeccion`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ProyeccionesUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ProyeccionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ProyeccionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES CON SELECT ===");
        var clientes = _unidad.Ordenes.ObtenerClientesUnicos();
        var resumenes = _unidad.Ordenes.ObtenerResumenes();
        var pendientes = _unidad.Ordenes.ObtenerResumenesPorEstado("Pendiente");
        var totales = _unidad.Ordenes.ObtenerOrdenesConTotales();
        if (clientes.Count != 3 || resumenes.Count != 5 || pendientes.Count != 3 || totales.Count != 5) throw new
        InvalidOperationException("Proyecciones inesperadas.");
        Console.WriteLine($"Clientes: {string.Join(", ", clientes)} | Resúmenes: {resumenes.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccion());
    }
}
```

CSHARP

Explicación línea a línea del caso de uso 3.3

Línea 1

`using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial `IUnidadDeTrabajo` e `IOrdenRepositorio`, que desacoplan el caso de uso de EF Core.

Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class ProyeccionesUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.
Línea 8	<code>public ProyeccionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	<code>Console.WriteLine("=== PROYECCIONES CON SELECT ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	<code>var clientes = _unidad.Ordenes.ObtenerClientesUnicos();</code> → Proyecta Cliente y elimina duplicados para comprobar una proyección escalar con Distinct.
Línea 14	<code>var resúmenes = _unidad.Ordenes.ObtenerResúmenes();</code> → Obtiene la proyección de las cinco órdenes a DTOs de resumen sin materializar entidades completas.
Línea 15	<code>var pendientes = _unidad.Ordenes.ObtenerResúmenesPorEstado("Pendiente");</code> → Combina filtro por estado y proyección a DTO para validar que ambas operaciones se ejecutan como una sola consulta.
Línea 16	<code>var totales = _unidad.Ordenes.ObtenerOrdenesConTotales();</code> → Proyecta cada orden junto con agregados de sus planchas para validar una proyección calculada sobre navegación.
Línea 17	<code>if (clientes.Count != 3 resúmenes.Count != 5 pendientes.Count != 3 totales.Count != 5) throw new InvalidOperationException("Proyecciones inesperadas.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	<code>Console.WriteLine(\$"Clientes: {string.Join(", ", clientes)} Resúmenes: {resúmenes.Count}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 19	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccion());</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 20	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 21	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
```

CSHARP

```

.SetBasePath(AppContext.BaseDirectory)
.AddJsonFile("appsettings.json", optional: false)
.AddEnvironmentVariables()
.Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ProyeccionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.3 OK");

```

Explicación línea a línea de Program.cs 3.3

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.

Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del composition root.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	<code>services.AddScoped<ProyeccionesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá AceriaDbContext, IUnidadDeTrabajo y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el DbContext para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.3 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

3.3 OK

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un tipo anónimo no puede actuar como contrato público entre capas; cuando el resultado cruza la frontera se usa un DTO.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.3

Diagnóstico técnico

Compara materializar la entidad completa con proyectar sólo `NumeroOrden`, `Cliente`, `Estado` y `FechaCreacion`. La ventaja demostrable es la reducción del shape transferido y de entidades materializadas. `Distinct()` debe permanecer en servidor si se aplica antes de `ToList()`.

Reto resuelto y verificación adicional

Usa `ObtenerClientesUnicos()` y confirma ausencia de duplicados y orden estable. Después contrasta `ObtenerResumenes()` con una carga completa: el DTO contiene exactamente los campos consumidos por Application.

Errores comunes revisados

Error	Causa	Solución
Propiedad no proyectada	Se accede a una propiedad que no está en la proyección	Proyectar todas las propiedades necesarias
Tipo anónimo fuera de ámbito	Se intenta usar un tipo anónimo fuera del método	Proyectar a un DTO con nombre
Función no traducida	Se usa una función que EF Core no puede traducir	Usar funciones traducibles
Proyección antes del filtro	Se proyecta antes de filtrar	Filtrar y ordenar antes de proyectar
Tipo de colección incompatible	El DTO exige <code>List<T></code> pero la proyección entrega otra forma de secuencia	Usar <code>ToList()</code> cuando el <i>shape</i> de destino exige una lista; no convertirlo en una regla universal
Distinct no aplicado	Se olvidó el operador <code>Distinct</code>	Añadir <code>.Distinct()</code> antes de <code>ToList</code>

Analogía operativa

Las proyecciones con `Select` en una acería son como los resúmenes que el jefe de planta pide al archivo central. En lugar de recibir la carpeta completa de cada orden con todos sus documentos, el jefe pide solo los datos que necesita: el número de orden, el cliente y el estado. El archivo central prepara un resumen con esos datos y lo entrega. El jefe no necesita los documentos completos para tomar decisiones: solo los datos clave. Proyectar a un tipo anónimo es

como pedir un resumen informal: se usa en el momento y no se archiva. Proyectar a un DTO es como pedir un formulario estandarizado: tiene nombre, se puede archivar y se puede usar en otros departamentos. Proyectar con `Distinct` es como pedir la lista de clientes únicos: sin duplicados. Proyectar con funciones de agregación es como pedir el total de planchas y el peso total: valores calculados en el archivo central. Así funcionan las proyecciones en EF Core: se seleccionan solo los datos necesarios, se reduce el volumen de información y se mejora el rendimiento.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Proyecta solo número de orden y cliente, y compara el `SELECT` con la carga de la entidad completa.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.3 OK, el siguiente estado parte exactamente de esta solución y añade **Proyecciones a DTOs**.

Punto 3.4 — Proyecciones a DTOs

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.3. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Proyecciones a DTOs** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.4
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida M2_2_12 Architecture. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerOrdenesConPlanchas`, `ObtenerOrdenesConDetalle`, `ObtenerOrdenesCompletas`, `ObtenerSqlProyeccionNavegacion`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ProyeccionesDtoUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ProyeccionesDtoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES A DTOs ===");
    }
}
```

CSHARP

```

var conPlanchas = _unidad.Ordenes.ObtenerOrdenesConPlanchas();
var conDetalle = _unidad.Ordenes.ObtenerOrdenesConDetalle();
var completas = _unidad.Ordenes.ObtenerOrdenesCompletas();
var primera = completas.Single(o => o.NumeroOrden == "OF-2024-0001");
if (conPlanchas.Count != 5 || conDetalle.Count != 5 || completas.Count != 5 || primera.Planchas.Count != 2
|| primera.Detalle is null) throw new InvalidOperationException("Proyecciones DTO inesperadas.");
Console.WriteLine($"OF-2024-0001 -> planchas: {primera.Planchas.Count}, detalle:
{primera.Detalle.ComposicionQuimica}");
Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccionNavegacion());
}
}

```

Explicación línea a línea del caso de uso 3.4

Línea 1	using AceriaData.Application.Interfaces; → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.
Línea 3	namespace AceriaData.Application.UseCases; → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	public sealed class ProyeccionesDtoUseCase → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.
Línea 8	public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	public void Ejecutar() → Define la operación docente que ejecutará el composition root.
Línea 11	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	Console.WriteLine("=== PROYECCIONES A DTOS ==="); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	var conPlanchas = _unidad.Ordenes.ObtenerOrdenesConPlanchas(); → Proyecta las órdenes a DTOs que contienen una colección anidada de planchas.
Línea 14	var conDetalle = _unidad.Ordenes.ObtenerOrdenesConDetalle(); → Proyecta cada orden junto con su DTO de detalle, contemplando que la navegación de referencia pueda ser nula.
Línea 15	var completas = _unidad.Ordenes.ObtenerOrdenesCompletas(); → Combina en un único DTO la información principal, la colección de planchas y el detalle de cada orden.
Línea 16	var primera = completas.Single(o => o.NumeroOrden == "OF-2024-0001"); → Selecciona de forma inequívoca la orden OF-2024-0001 del resultado proyectado para comprobar sus datos anidados.
Línea 17	if (conPlanchas.Count != 5 conDetalle.Count != 5 completas.Count != 5 primera.Planchas.Count != 2 primera.Detalle is null) throw new InvalidOperationException("Proyecciones DTO inesperadas."); → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	Console.WriteLine(\$"OF-2024-0001 -> planchas: {primera.Planchas.Count}, detalle: {primera.Detalle.ComposicionQuimica}"); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 19	Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccionNavegacion()); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 20	} → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21

} → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB");
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ProyeccionesDtoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.4 OK");

```

Explicación línea a línea de Program.cs 3.4

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Inicia la construcción de la configuración de la aplicación.

Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Fija como base el directorio de ejecución para localizar <code>appsettings.json</code> .
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer <code>ConnectionStrings:AceriaDB</code> .
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Obtiene la cadena <code>AceriaDB</code> desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión <code>AceriaDB</code> , evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del <code>composition root</code> .
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra <code>DbContext</code> , repositorio y unidad de trabajo de <code>Infrastructure</code> .
Línea 20	<code>services.AddScoped<ProyeccionesDtoUseCase>();</code> → Registra el caso de uso con ciclo de vida <code>Scoped</code> para compartir el mismo ámbito que <code>DbContext</code> .
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios <code>IDisposable</code> al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio <code>Scoped</code> consumido incorrectamente desde un <code>Singleton</code> .
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito <code>Scoped</code> que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.

Línea 38

`Console.WriteLine("3.4 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.4 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: No comprobar una relación opcional antes de proyectarla rompe la semántica de null del modelo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.4

Diagnóstico técnico

Las proyecciones a DTO deben decidir qué ocurre con relaciones opcionales. `Detalle` puede ser `null`; el DTO también debe admitirlo. Las colecciones relacionadas se convierten a DTOs de lectura, no se filtran entidades de Infrastructure hacia Application.

Reto resuelto y verificación adicional

Selecciona `0F-2024-0001` en `OrdenCompletaDto`: debe tener dos planchas y detalle. Comprueba también una relación opcional ausente sin provocar `NullReferenceException`.

Errores comunes revisados

Error	Causa	Solución
<code>NullReferenceException</code> en DTO anidado	No se comprobó si la entidad relacionada es null	Usar operador ternario antes de proyectar
Tipo de colección incompatible	El DTO declara <code>List<T></code> pero la proyección no produce una lista compatible	Materializar con <code>ToList()</code> cuando lo exija el tipo de destino
Propiedad no proyectada	Se accede a una propiedad que no está en la proyección	Proyectar todas las propiedades necesarias

Error	Causa	Solución
GroupBy tratado siempre como GROUP BY SQL	Se presupone que cualquier forma de agrupación tiene la misma traducción	Para el patrón relacional usar clave + agregados; para otros <i>shapes</i> , inspeccionar la traducción real
SelectMany con colección vacía	La entidad principal no aparece en el resultado	Usar DefaultIfEmpty
Average sobre colección vacía	La semántica depende de si el tipo es anulable	Proyectar a nullable y definir el resultado, o validar que existan elementos

Analogía operativa

Las proyecciones a DTOs en una acería son como los formularios estandarizados que se usan para transferir información entre departamentos. En lugar de enviar la carpeta completa de una orden con todos sus documentos, se envía un formulario con los campos que el departamento receptor necesita. El formulario tiene un nombre, un formato fijo y se puede archivar. Proyectar a un DTO con inicializador de objeto es como rellenar un formulario campo a campo. Proyectar a un DTO con constructor es como usar un formulario preimpreso que garantiza que todos los campos estén rellenos. Proyectar una colección de navegación dentro de un DTO es como adjuntar una lista de planchas al formulario de la orden. Proyectar un DTO anidado es como incluir un formulario de detalle dentro del formulario principal. SelectMany es como aplanar varias listas en una sola. GroupBy con agregaciones es como pedir un resumen por cliente con totales. Así funcionan las proyecciones a DTOs en EF Core: se seleccionan solo los datos necesarios, se estructuran en formularios estandarizados y se transfieren entre capas de forma eficiente.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Crea un DTO que incluya una lista de planchas y un detalle opcional sin exponer entidades.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.4 OK, el siguiente estado parte exactamente de esta solución y añade **Consultas de agregación: Count, Sum, Average, Min y Max**.

Punto 3.5 — Consultas de agregación: Count, Sum, Average, Min y Max

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.4. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Consultas de agregación: Count, Sum, Average, Min y Max** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.5
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ContarOrdenes`, `ContarOrdenesPorEstado`, `ExisteAlgunaOrden`, `TodasLasOrdenesTienenEstado`, `ObtenerPesoTotalDePlanchas`, `ObtenerPesoPromedioDePlanchas`, `ObtenerPesoMinimoDePlanchas`, `ObtenerPesoMaximoDePlanchas`, `ObtenerResumenPorCliente`, `ObtenerResumenPorEstado` y `ObtenerResumenMensual`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/AgregacionesUseCase.cs`

CSHARP

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class AgregacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public AgregacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== AGREGACIONES ===");
        var total = _unidad.Ordenes.ContarOrdenes();
        var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");
        var existe = _unidad.Ordenes.ExisteAlgunaOrden();
        var todasConEstado = _unidad.Ordenes.TodasLasOrdenesTienenEstado();
        var peso = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();
        var promedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();
        var minimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();
        var maximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();
        var porCliente = _unidad.Ordenes.ObtenerResumenPorCliente();
        var porEstado = _unidad.Ordenes.ObtenerResumenPorEstado();
        var mensual = _unidad.Ordenes.ObtenerResumenMensual();
        if (total != 5 || pendientes != 3 || !existe || !todasConEstado || peso != 1426.9m || promedio != 285.38m ||
            minimo != 125.6m || maximo != 450.0m || porCliente.Count != 3 || porEstado.Count != 3 || mensual.Count != 5) throw
            new InvalidOperationException("Agregaciones inesperadas.");
        Console.WriteLine($"Órdenes: {total} | Pendientes: {pendientes} | Peso: {peso:N1} kg | Promedio:
{promedio:N2} kg | Min: {minimo:N1} kg | Max: {maximo:N1} kg");
    }
}
```

Explicación línea a línea del caso de uso 3.5

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application que permiten consumir el repositorio sin referenciar EF Core desde el caso de uso.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el caso de uso dentro de la capa Application.
Línea 5	<code>public sealed class AgregacionesUseCase</code> → Declara el caso de uso que valida las agregaciones del punto 3.5.
Línea 6	<code>{</code> → Abre el bloque de la clase.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Conserva la unidad de trabajo inyectada para acceder al repositorio.
Línea 8	<code>public AgregacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la dependencia mediante inyección de constructor.
Línea 10	<code>public void Ejecutar()</code> → Define la operación que ejecuta todas las agregaciones y sus aserciones E2E.
Línea 11	<code>{</code> → Abre el cuerpo del método Ejecutar.
Línea 12	<code>Console.WriteLine("=== AGREGACIONES ===");</code> → Identifica en consola el bloque de pruebas de agregación.

Línea 13	<code>var total = _unidad.Ordenes.ContarOrdenes();</code> → Ejecuta Count sobre las órdenes y conserva el total para validarlo.
Línea 14	<code>var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");</code> → Ejecuta Count con predicado para contar únicamente las órdenes pendientes.
Línea 15	<code>var existe = _unidad.Ordenes.ExisteAlgunaOrden();</code> → Ejecuta Any para comprobar existencia sin solicitar un recuento total.
Línea 16	<code>var todasConEstado = _unidad.Ordenes.TodasLasOrdenesTienenEstado();</code> → Ejecuta All para verificar que todas las órdenes cumplen la condición de tener estado no vacío.
Línea 17	<code>var peso = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();</code> → Ejecuta Sum sobre los pesos y obtiene el total determinista del dataset.
Línea 18	<code>var promedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();</code> → Ejecuta Average sobre los pesos proyectados como anulables, con valor cero definido para una colección vacía.
Línea 19	<code>var minimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();</code> → Ejecuta Min y obtiene el peso mínimo de las planchas sembradas.
Línea 20	<code>var maximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();</code> → Ejecuta Max y obtiene el peso máximo de las planchas sembradas.
Línea 21	<code>var porCliente = _unidad.Ordenes.ObtenerResumenPorCliente();</code> → Agrupa por cliente y materializa los resúmenes agregados de cada grupo.
Línea 22	<code>var porEstado = _unidad.Ordenes.ObtenerResumenPorEstado();</code> → Agrupa por estado y materializa un resumen por cada estado presente.
Línea 23	<code>var mensual = _unidad.Ordenes.ObtenerResumenMensual();</code> → Agrupa por año y mes para validar GroupBy con claves temporales y agregados.
Línea 24	<code>if (total != 5 pendientes != 3 !existe !todasConEstado peso != 1426.9m promedio != 285.38m minimo != 125.6m maximo != 450.0m porCliente.Count != 3 porEstado.Count != 3 mensual.Count != 5) throw new InvalidOperationException("Agregaciones inesperadas.");</code> → Contrasta Count, Any, All, Sum, Average, Min, Max y los GroupBy contra el dataset determinista; cualquier desviación hace fallar el E2E.
Línea 25	<code>Console.WriteLine(\$"Órdenes: {total} Pendientes: {pendientes} Peso: {peso:N1} kg Promedio: {promedio:N2} kg Min: {minimo:N1} kg Max: {maximo:N1} kg");</code> → Publica los valores agregados validados para facilitar la inspección manual de la ejecución.
Línea 26	<code>}</code> → Cierra el cuerpo del método Ejecutar.
Línea 27	<code>}</code> → Cierra la clase AgregacionesUseCase.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();
```

CSHARP

```

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<AgregacionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.5 OK");

```

Explicación línea a línea de Program.cs 3.5

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.
Línea 16	?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del composition root.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	<code>services.AddScoped<AgregacionesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios IDisposable al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el DbContext para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.5 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

3.5 0K

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Average/Min/Max sobre una secuencia vacía requieren una estrategia explícita; el repositorio usa proyección nullable y coalescencia.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.5

Diagnóstico técnico

Para agregaciones sobre conjuntos potencialmente vacíos, el repositorio proyecta a `decimal?` y aplica `?? 0m`. `Any()` expresa existencia y no debe sustituirse mecánicamente por `Count() > 0`.

Reto resuelto y verificación adicional

Calcula cardinalidades y peso total/promedio/mínimo/máximo y revisa que el SQL use agregados del servidor. Después agrupa por cliente y estado y contrasta los resultados con el dataset determinista.

Errores comunes revisados

Error	Causa	Solución
Average sobre colección vacía	Una sobrecarga no anulable puede fallar; una anulable puede devolver null	En <code>AceriaData</code> proyectar a <code>decimal?</code> y aplicar <code>?? 0m</code>
Min o Max sobre colección vacía	El resultado no anulable no tiene un valor válido que devolver	Proyectar a nullable y definir el valor del contrato, o comprobar existencia
GroupBy sin agregación	Se intenta tratar cualquier agrupación como <code>GROUP BY SQL</code>	Usar claves + agregados para la traducción relacional o revisar el <i>shape</i> real
HAVING mal expresado	El filtro agregado se aplica a las filas antes de agrupar	Aplicar <code>Where</code> sobre el agrupamiento para expresar el filtro de grupos
Count después de <code>ToList</code>	Se transfirieron filas solo para contarlas después en memoria	Si solo se necesita el total, llamar a <code>Count()</code> directamente sobre <code>IQueryable</code>
Agregación vacía	No se definió la semántica del resultado	Usar nullable + <code>??</code> cuando el contrato de <code>AceriaData</code> requiera un valor por defecto
Any con <code>Count() > 0</code>	Se usa <code>Count</code> en lugar de <code>Any</code>	Usar <code>Any</code> para mejor rendimiento

Analogía operativa

Las consultas de agregación en una acería son como los informes que el jefe de planta pide al archivo central. En lugar de revisar todas las carpetas una por una, el jefe pide un resumen: cuántas órdenes hay, cuánto pesan las planchas, cuál es la fecha más antigua. El archivo central calcula el resumen y lo entrega. `Count` es como contar las carpetas. `Sum` es como sumar los pesos. `Average` es como calcular el promedio. `Min` y `Max` son como encontrar el más ligero y el más pesado. `GroupBy` es como agrupar las carpetas por cliente o por estado. El filtro `Having` es como pedir solo los grupos que cumplen una condición. Las agregaciones sobre colecciones vacías son como pedir un promedio cuando no hay datos: hay que tener cuidado porque puede dar error. `DefaultIfEmpty` introduce un elemento por

defecto en una secuencia vacía; el valor concreto y su traducción deben analizarse según el tipo y el proveedor. Así funcionan las agregaciones en EF Core: se calculan en el servidor, se devuelven resúmenes y se evita transferir todas las filas.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Calcula un resumen mensual y explica qué operaciones se ejecutan como agregados SQL.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.5 OK, el siguiente estado parte exactamente de esta solución y añade **Agrupaciones con proyección**.

Punto 3.6 — Agrupaciones con proyección

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.5. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Agrupaciones con proyección** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.6
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerResumenPorClienteConOrdenes`, `ObtenerResumenPorClienteYEstado`, `ObtenerResumenPorClienteYEstadoConFiltro`, `ObtenerResumenMensualConOrdenes`, `ObtenerSqlAgrupacionClienteEstado`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/AgrupacionesUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class AgrupacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public AgrupacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");
        var porCliente = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();
    }
}
```

CSHARP

```

var porClienteEstado = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();
var having = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();
var mensual = _unidad.Ordenes.ObtenerResumenMensualConOrdenes();
var norte = porCliente.Single(x => x.Cliente == "Constructora del Norte");
if (norte.TotalOrdenes != 3 || norte.Ordenes.Count != 3 || porClienteEstado.Count != 4 || having.Count != 1
|| mensual.Count != 5) throw new InvalidOperationException("Agrupaciones inesperadas.");
Console.WriteLine($"Norte: {norte.TotalOrdenes} órdenes | HAVING: {having.Count} grupo");
Console.WriteLine(_unidad.Ordenes.ObtenerSqlAgrupacionClienteEstado());
}
}

```

Explicación línea a línea del caso de uso 3.6

Línea 1	using AceriaData.Application.Interfaces; → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.
Línea 3	namespace AceriaData.Application.UseCases; → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	public sealed class AgrupacionesUseCase → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.
Línea 8	public AgrupacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	public void Ejecutar() → Define la operación docente que ejecutará el composition root.
Línea 11	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ==="); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	var porCliente = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes(); → Obtiene cabeceras agrupadas por cliente y las órdenes de cada grupo con la estrategia acotada implementada en Infrastructure.
Línea 14	var porClienteEstado = _unidad.Ordenes.ObtenerResumenPorClienteYEstado(); → Agrupa por la clave compuesta Cliente/Estado y materializa los agregados de cada combinación.
Línea 15	var having = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro(); → Aplica un filtro sobre el resultado agrupado para demostrar el patrón equivalente a HAVING.
Línea 16	var mensual = _unidad.Ordenes.ObtenerResumenMensualConOrdenes(); → Agrupa por año y mes y compone los resúmenes mensuales con las órdenes pertenecientes a cada grupo.
Línea 17	var norte = porCliente.Single(x => x.Cliente == "Constructora del Norte"); → Localiza el único resumen de Constructora del Norte para validar el número de órdenes y la colección interna proyectada.
Línea 18	if (norte.TotalOrdenes != 3 norte.Ordenes.Count != 3 porClienteEstado.Count != 4 having.Count != 1 mensual.Count != 5) throw new InvalidOperationException("Agrupaciones inesperadas."); → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 19	Console.WriteLine(\$"Norte: {norte.TotalOrdenes} órdenes HAVING: {having.Count} grupo"); → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlAgrupacionClienteEstado());</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 21	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 22	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<AgrupacionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.6 OK");
```

Explicación línea a línea de Program.cs 3.6

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7	<code>using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.</code>
Línea 9	<code>var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.</code>
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.</code>
Línea 12	<code>.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.</code>
Línea 13	<code>.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.</code>
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.</code>
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.</code>
Línea 18	<code>var services = new ServiceCollection(); → Crea el contenedor de servicios del composition root.</code>
Línea 19	<code>services.AddAceriaInfrastructure(connectionString); → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.</code>
Línea 20	<code>services.AddScoped<AgrupacionesUseCase>(); → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.</code>
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.</code>
Línea 23	<code>{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.</code>
Línea 24	<code>ValidateOnBuild = true, → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.</code>
Línea 25	<code>ValidateScopes = true → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.</code>
Línea 26	<code>}); → Cierra las opciones y completa la llamada que construye el proveedor de servicios.</code>
Línea 27	<code>using var scope = provider.CreateScope(); → Crea el ámbito Scoped que compartirá AceriaDbContext, IUnidadDeTrabajo y el caso de uso durante esta ejecución.</code>
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → Resuelve el DbContext para preparar la base de demostración.</code>
Línea 30	<code>context.Database.EnsureDeleted(); → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.</code>
Línea 31	<code>context.Database.Migrate(); → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.</code>
Línea 32	<code>DemoData.Seed(context); → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.</code>

Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.6 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.6 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.6, la colección interna por cliente se resuelve con **dos consultas acotadas** y composición posterior en memoria. Esto evita enseñar que una proyección de colección dentro de `GroupBy` implique necesariamente una consulta adicional por grupo. El SQL de `GROUP BY` y `HAVING` se valida con una consulta agregada independiente.

Paso 9: Diagnosticar un error común

Error: No hay que asumir que una colección interna dentro de `GroupBy` implica automáticamente N+1. Se debe inspeccionar la consulta real y diseñar una estrategia acotada.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.6

Diagnóstico técnico

Una colección interna dentro de `GroupBy` no implica automáticamente N+1. El checkpoint usa una consulta de cabeceras agrupadas y otra de filas proyectadas, y compone en memoria con un número fijo de consultas. `Where(g => g.Count() > 1)` debe observarse como `HAVING`.

Reto resuelto y verificación adicional

Comprueba tres órdenes para Constructora del Norte y un único grupo cliente-estado con más de una orden. `ObtenerSqlAgrupacionClienteEstado()` debe contener un `GROUP BY` real.

Errores comunes revisados

Error	Causa	Solución
Colección interna agrupada	Se presupuso una traducción SQL o N+1 sin medirla	Inspeccionar la traducción o usar la estrategia acotada de dos consultas de 3.6
Filtro colocado en la fase equivocada	<code>Where</code> antes de <code>GroupBy</code> filtra filas; <code>Where</code> sobre el grupo filtra grupos	Elegir conscientemente <code>WHERE</code> o <code>HAVING</code> según lo que se quiera filtrar
Proyección relacional de grupo no válida	Se intenta proyectar una columna que no forma parte de la clave ni de un agregado en un <code>GROUP BY SQL</code>	Usar la clave/agregados o cambiar el <i>shape</i> y comprobar su traducción
N+1 asumido en agrupaciones	Se atribuye N+1 a cualquier colección interna sin medir consultas	Inspeccionar la traducción real y usar una estrategia acotada como la de 3.6
SelectMany sin proyección	Se aplanar sin proyectar	Añadir una proyección clara
Agrupación anidada	Se presupuso una traducción o número de consultas fijo	Revisar el <i>shape</i> y hacer explícita la frontera cliente/servidor si hace falta

Analogía operativa

Las agrupaciones con proyección en una acería son como los informes agrupados que el jefe de planta pide al archivo central. En lugar de una lista plana de órdenes, el jefe pide un informe agrupado por cliente: para cada cliente, cuántas órdenes tiene y cuáles son. El archivo central agrupa las carpetas por cliente, cuenta las órdenes y prepara una lista con los números de cada orden. Proyectar una colección interna dentro de un grupo es como adjuntar la lista de órdenes al informe del cliente. El filtro Having es como pedir solo los clientes que tienen más de una orden. La agrupación por múltiples claves es como agrupar por cliente y estado a la vez. SelectMany es como aplanar los grupos en una lista plana. Así funcionan las agrupaciones con proyección en EF Core: se agrupan los datos en el servidor, se proyectan los resúmenes y se adjuntan las colecciones internas cuando es necesario. El resultado es un informe estructurado y eficiente.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Construye un HAVING para grupos con más de una orden y verifica que el SQL contiene GROUP BY/HAVING.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.6 OK, el siguiente estado parte exactamente de esta solución y añade **Joins y navegación en consultas**.

Punto 3.7 — Joins y navegación en consultas

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.6. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Joins y navegación en consultas** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.7
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerJoinOrdenesPlanchas`, `ObtenerLeftJoinOrdenesPlanchas`, `ObtenerOrdenesConDetalleJoin`, `ObtenerOrdenesConAleaciones`, `ObtenerSqlJoinExplicito`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/JoinsUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class JoinsUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public JoinsUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");
        var inner = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();
        var left = _unidad.Ordenes.ObtenerLeftJoinOrdenesPlanchas();
        var detalle = _unidad.Ordenes.ObtenerOrdenesConDetalleJoin();
        var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleaciones();
        if (inner.Count != 5 || left.Count != 6 || detalle.Count != 5 || aleaciones.Count != 5) throw new
InvalidOperationException("Joins inesperados.");
        if (!left.Any(x => x.NumeroOrden == "OF-2024-0004" && x.Peso is null)) throw new
InvalidOperationException("LEFT JOIN no conservó la orden sin planchas.");
        Console.WriteLine($"INNER filas: {inner.Count} | LEFT filas: {left.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlJoinExplicito());
    }
}
```

CSHARP

Explicación línea a línea del caso de uso 3.7

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application, en especial <code>IUnidadDeTrabajo</code> e <code>IOrdenRepositorio</code> , que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class JoinsUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de <code>DbContext</code> .
Línea 8	<code>public JoinsUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12	<code>Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	<code>var inner = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();</code> → Ejecuta el INNER JOIN explícito entre órdenes y planchas y conserva una fila por coincidencia.
Línea 14	<code>var left = _unidad.Ordenes.ObtenerLeftJoinOrdenesPlanchas();</code> → Ejecuta el patrón de LEFT JOIN con <code>DefaultIfEmpty</code> para conservar también las órdenes sin planchas.
Línea 15	<code>var detalle = _unidad.Ordenes.ObtenerOrdenesConDetalleJoin();</code> → Combina órdenes y detalle mediante navegación/proyección para validar una relación de referencia opcional.
Línea 16	<code>var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleaciones();</code> → Proyecta órdenes junto con sus relaciones de aleación para comprobar la navegación a través de la entidad intermedia.
Línea 17	<code>if (inner.Count != 5 left.Count != 6 detalle.Count != 5 aleaciones.Count != 5)</code> <code>throw new InvalidOperationException("Joins inesperados.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	<code>if (!left.Any(x => x.NumeroOrden == "0F-2024-0004" && x.Peso is null)) throw new</code> <code>InvalidOperationException("LEFT JOIN no conservó la orden sin planchas.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 19	<code>Console.WriteLine(\$"INNER filas: {inner.Count} LEFT filas: {left.Count}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 20	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlJoinExplicito());</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 21	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 22	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<JoinsUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true,
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();
```

```
var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.7 OK");
```

Explicación línea a línea de Program.cs 3.7

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.
Línea 16	?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	var services = new ServiceCollection(); → Crea el contenedor de servicios del composition root.
Línea 19	services.AddAceriaInfrastructure(connectionString); → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	services.AddScoped<JoinsUseCase>(); → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	using var provider = services.BuildServiceProvider(new ServiceProviderOptions → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.
Línea 23	{ → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24	<code>ValidateOnBuild = true</code> , → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.7 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infraestructura o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.7 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un INNER JOIN elimina órdenes sin coincidencia. Si deben conservarse, se usa un patrón LEFT JOIN y propiedades anulables.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.7

Diagnóstico técnico

`Join` modela coincidencias; el patrón de `left join` conserva la fila exterior y exige nulabilidad en la proyección interior. Con varias relaciones uno-a-muchos puede multiplicarse el número de filas: no son duplicados accidentales, sino el shape relacional.

Reto resuelto y verificación adicional

Compara `ObtenerJoinOrdenesPlanchas()` y `ObtenerLeftJoinOrdenesPlanchas()`: la orden sin planchas desaparece del INNER JOIN y permanece en el LEFT JOIN con datos relacionados nulos. Revisa la condición por `OrdenId` en el SQL.

Errores comunes revisados

Error	Causa	Solución
NullReferenceException en detalle	No se comprobó si el detalle es null	Usar operador ternario antes de proyectar
Se perdieron filas sin relación	En el patrón correlacionado se aplanó la colección sin <code>DefaultIfEmpty()</code>	Usar el patrón de <code>LEFT JOIN</code> con <code>DefaultIfEmpty()</code> cuando deban conservarse las entidades exteriores
Multiplicación de filas	Se cargaron colecciones hermanas con <code>JOIN</code> al mismo nivel	Evaluar proyección o <code>AsSplitQuery()</code> según el <code>shape</code>
ThenInclude sin Include	Se usó <code>ThenInclude</code> sin <code>Include</code> previo	Usar <code>Include</code> antes de <code>ThenInclude</code>
GroupJoin con forma no traducible	Se devolvió directamente la agrupación sin respetar un patrón soportado	Para un <code>LEFT JOIN</code> , aplanar inmediatamente con <code>DefaultIfEmpty()</code> y comprobar la traducción
Proyección de colección no validada	Se asumió que añadir <code>ToList()</code> determina la traducción o elimina N+1	Inspeccionar la traducción y reformular el <code>shape</code> ; <code>ToList()</code> no es una cura universal para N+1

Analogía operativa

Los joins en una acería son como las consultas que el jefe de planta hace al archivo central para combinar información de varias carpetas. En lugar de mirar la carpeta de órdenes, la carpeta de planchas y la carpeta de aleaciones por separado, el jefe pide un informe que combine las tres. El `Join` explícito es como indicar manualmente cómo se relacionan las carpetas; el patrón `GroupJoin + DefaultIfEmpty` permite conservar la carpeta principal aunque no exista una relacionada. Las propiedades de navegación permiten expresar relaciones sin escribir el join manualmente. En EF Core, la forma SQL concreta depende del patrón: puede aparecer un `JOIN`, una subconsulta u otra construcción, y la carga relacionada con `AsSplitQuery()` puede dividirse en varias sentencias coordinadas. Por eso el laboratorio inspecciona el SQL real en lugar de asumir una única forma.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Incluye órdenes sin planchas con `LEFT JOIN` y demuestra que OF-2024-0004 sigue presente.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.7 OK, el siguiente estado parte exactamente de esta solución y añade **Eager Loading con Include y ThenInclude**.

Punto 3.8 — Eager Loading con Include y ThenInclude

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.7. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Eager Loading con Include y ThenInclude** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.8
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerOrdenesConPlanchasInclude`, `ObtenerOrdenesConPlanchasYDetalleInclude`, `ObtenerOrdenesConAleacionesInclude`, `ObtenerOrdenesConPlanchasPesadasInclude`, `ObtenerOrdenesConPlanchasYDetalleSplitQuery`, `ObtenerSqlInclude`, `ObtenerOrdenesAutoInclude` y `ObtenerOrdenesIgnorandoAutoInclude`. El punto demuestra además la configuración `AutoInclude` y su neutralización por consulta mediante `IgnoreAutoIncludes`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaEagerUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CargaEagerUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaEagerUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== EAGER LOADING ===");
        var planchas = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
        var detalle = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude();
        var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude();
        var filtradas = _unidad.Ordenes.ObtenerOrdenesConPlanchasPesadasInclude();
        var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();
        var auto = _unidad.Ordenes.ObtenerOrdenesAutoInclude();
        var sinAuto = _unidad.Ordenes.ObtenerOrdenesIgnorandoAutoInclude();

        if (planchas.Count != 5 || detalle.Count != 5 || aleaciones.Count != 5 || split.Count != 5)
            throw new InvalidOperationException("Carga Eager inesperada.");
        if (filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 1)
            throw new InvalidOperationException("Filtered Include inesperado.");
        if (auto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 2)
            throw new InvalidOperationException("AutoInclude no cargó Planchas.");
    }
}
```

CSHARP

```

if (sinAuto.Single(o => o.NumeroOrden == "0F-2024-0001").Planchas.Count != 0)
    throw new InvalidOperationException("IgnoreAutoIncludes no suprimió la carga automática.");
if (split.Sum(o => o.Planchas.Count) != 5 || split.Sum(o => o.OrdenesAleaciones.Count) != 4)
    throw new InvalidOperationException("SplitQuery no materializó las dos colecciones esperadas.");

Console.WriteLine($"Include: {planchas.Count} órdenes | SplitQuery: {split.Count} | AutoInclude: {auto.Count} | IgnoreAutoIncludes: {sinAuto.Count}");
    Console.WriteLine(_unidad.Ordenes.ObtenerSqlInclude());
}
}

```

Explicación línea a línea del caso de uso 3.8

Línea 1	using AceriaData.Application.Interfaces; → Importa IUnidadDeTrabajo, el puerto que permite al caso de uso consumir consultas sin conocer AceriaDbContext.
Línea 3	namespace AceriaData.Application.UseCases; → Sitúa la clase en el espacio de nombres de casos de uso de Application.
Línea 5	public sealed class CargaEagerUseCase → Declara el caso de uso dedicado a comparar las variantes de Eager Loading del checkpoint.
Línea 6	{ → Abre el bloque de la declaración inmediatamente anterior.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Conserva la unidad de trabajo inyectada para acceder al repositorio de órdenes.
Línea 8	public CargaEagerUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Recibe la unidad de trabajo mediante DI y la asigna al campo de solo lectura.
Línea 10	public void Ejecutar() → Define la operación que ejecutará todas las comprobaciones E2E de carga Eager.
Línea 11	{ → Abre el bloque de la declaración inmediatamente anterior.
Línea 12	Console.WriteLine("=== EAGER LOADING ==="); → Muestra la cabecera de la demostración.
Línea 13	var planchas = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude(); → Ejecuta Include sobre la colección Planchas.
Línea 14	var detalle = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude(); → Carga conjuntamente Planchas y la referencia Detalle.
Línea 15	var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude(); → Ejecuta Include + ThenInclude sobre OrdenesAleaciones → Aleacion.
Línea 16	var filtradas = _unidad.Ordenes.ObtenerOrdenesConPlanchasPesadasInclude(); → Ejecuta Filtered Include y conserva las planchas que cumplen el umbral del repositorio.
Línea 17	var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery(); → Ejecuta AsSplitQuery sobre Planchas y OrdenesAleaciones, más Detalle, para evitar expansión cartesiana entre colecciones hermanas.
Línea 18	var auto = _unidad.Ordenes.ObtenerOrdenesAutoInclude(); → Consulta órdenes sin Include explícito; el modelo debe cargar Planchas automáticamente mediante AutoInclude.
Línea 19	var sinAuto = _unidad.Ordenes.ObtenerOrdenesIgnorandoAutoInclude(); → Ejecuta la lectura con IgnoreAutoIncludes para demostrar que la carga automática puede suprimirse por consulta.
Línea 21	if (planchas.Count != 5 detalle.Count != 5 aleaciones.Count != 5 split.Count != 5) → Comprueba que todas las estrategias devuelven las cinco órdenes visibles del dataset.
Línea 22	throw new InvalidOperationException("Carga Eager inesperada."); → Hace fallar el E2E si la forma principal de la carga no coincide con el dataset.

Línea 23	<code>if (filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 1) → Verifica que Filtered Include deja una sola plancha de al menos 300 kg en OF-2024-0001.</code>
Línea 24	<code>throw new InvalidOperationException("Filtered Include inesperado."); → Falla si el filtro de la colección incluida no se respeta.</code>
Línea 25	<code>if (auto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 2) → Comprueba que AutoInclude cargó las dos planchas de OF-2024-0001 sin Include explícito.</code>
Línea 26	<code>throw new InvalidOperationException("AutoInclude no cargó Planchas."); → Falla si la navegación configurada con AutoInclude no fue materializada.</code>
Línea 27	<code>if (sinAuto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 0) → Comprueba que IgnoreAutoIncludes deja la colección sin cargar en una consulta AsNoTracking.</code>
Línea 28	<code>throw new InvalidOperationException("IgnoreAutoIncludes no suprimió la carga automática."); → Falla si la consulta no logra neutralizar AutoInclude.</code>
Línea 29	<code>if (split.Sum(o => o.Planchas.Count) != 5 split.Sum(o => o.OrdenesAleaciones.Count) != 4) → Valida que SplitQuery materializó las cinco planchas y las cuatro relaciones de aleación del dataset.</code>
Línea 30	<code>throw new InvalidOperationException("SplitQuery no materializó las dos colecciones esperadas."); → Falla si alguna de las dos colecciones incluidas queda incompleta.</code>
Línea 32	<code>Console.WriteLine(\$"Include: {planchas.Count} órdenes SplitQuery: {split.Count} AutoInclude: {auto.Count} IgnoreAutoIncludes: {sinAuto.Count}"); → Resume en consola los tamaños obtenidos por las estrategias validadas.</code>
Línea 33	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlInclude()); → Imprime el SQL generado para la consulta Include de referencia.</code>
Línea 34	<code>} → Cierra el bloque de la declaración inmediatamente anterior.</code>
Línea 35	<code>} → Cierra el bloque de la declaración inmediatamente anterior.</code>

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaEagerUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

```

context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.8 OK");

```

Explicación línea a línea de Program.cs 3.8

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	.AddEnvironmentVariables() → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	.Build(); → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Obtiene la cadena AceriaDB desde la configuración.
Línea 16	?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	var services = new ServiceCollection(); → Crea el contenedor de servicios del composition root.
Línea 19	services.AddAceriaInfrastructure(connectionString); → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	services.AddScoped<CargaEagerUseCase>(); → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	using var provider = services.BuildServiceProvider(new ServiceProviderOptions → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.

Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.8 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.8 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.8, el Filtered Include usa `AsNoTracking()` para que navigation fix-up no reincorpore planchas previamente rastreadas que no cumplan el filtro.

Paso 9: Diagnosticar un error común

Error: Filtered Include con entidades ya rastreadas puede verse afectado por navigation fix-up; el ejemplo usa `AsNoTracking`.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.8

Diagnóstico técnico

El checkpoint ejecuta `Include`, `ThenInclude`, `Filtered Include`, `AsSplitQuery()`, `AutoInclude()` e `IgnoreAutoIncludes()`. El split carga dos colecciones hermanas —Planchas y OrdenesAleaciones— más Detalle, de modo que la decisión responde a un riesgo real de multiplicación de filas.

Reto resuelto y verificación adicional

Valida OF-2024-0001 con dos planchas mediante `AutoInclude`, cero planchas usando `IgnoreAutoIncludes` + `AsNoTracking`, una sola plancha en `Filtered Include` y, en `SplitQuery`, cinco planchas y cuatro relaciones de aleación en todo el dataset.

Errores comunes revisados

Error	Causa	Solución
Multiplicación de filas	Se cargaron colecciones hermanas al mismo nivel en modo single-query	Comparar <code>AsSingleQuery()</code> y <code>AsSplitQuery()</code> para el volumen real
N+1	Se accedió repetidamente a navegaciones Lazy no cargadas	Planificar Eager Loading, proyección o Explicit Loading según el caso
ThenInclude sin Include	Se usó ThenInclude sin Include previo	Usar Include antes de ThenInclude
NullReferenceException en referencia	La propiedad de navegación es null	Comprobar si la propiedad es null
Filtro no aplicado	Se aplicó el filtro fuera del Include	Aplicar el filtro dentro del Include
AutoInclude excesivo	Se configuró en muchas propiedades	Usar con moderación

Analogía operativa

La carga Eager en una acería es como pedirle al archivo central que, además de la carpeta de la orden, adjunte también las carpetas de las planchas y del detalle. En lugar de pedir la carpeta de la orden y después ir a buscar las planchas y el detalle por separado, se pide todo junto. El archivo central prepara un paquete con la orden y sus documentos relacionados. Include es como pedir que se adjunte una carpeta. ThenInclude es como pedir que se adjunten los documentos de la carpeta adjunta. Filtered Include es como pedir que solo se adjunten las planchas activas. AsSplitQuery es como pedir que el paquete se prepare en varios envíos separados para evitar que el paquete sea demasiado grande. AutoInclude es como pedir que siempre se adjunten las planchas, sin tener que pedirlo cada vez. Así funciona la carga Eager en EF Core: las relaciones necesarias se solicitan de forma planificada como parte de la operación inicial, que puede resolverse con una consulta única o con varias consultas coordinadas mediante `AsSplitQuery()`.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Compara Include único y AsSplitQuery con las mismas relaciones y observa el SQL.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.8 OK, el siguiente estado parte exactamente de esta solución y añade **Lazy Loading: configuración, funcionamiento y riesgos**.

Punto 3.9 — Lazy Loading: configuración, funcionamiento y riesgos

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.8. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Lazy Loading: configuración, funcionamiento y riesgos** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.9
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerTodasSinInclude`, `IgnoreAutoIncludes`, `UseLazyLoadingProxies`, `virtual`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaLazyUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CargaLazyUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaLazyUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== LAZY LOADING (DEMOSTRACIÓN) ===");
        var ordenes = _unidad.Ordenes.ObtenerTodasSinInclude();
        var totalPlanchas = 0;
        foreach (var orden in ordenes)
            totalPlanchas += orden.Planchas.Count;
        if (ordenes.Count != 5 || totalPlanchas != 5) throw new InvalidOperationException("Lazy Loading no cargó las relaciones.");
        Console.WriteLine($"Órdenes: {ordenes.Count} | Planchas accedidas bajo demanda: {totalPlanchas}");
        Console.WriteLine("Advertencia docente: el acceso dentro de un bucle puede producir N+1 consultas.");
    }
}
```

CSHARP

Explicación línea a línea del caso de uso 3.9

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class CargaLazyUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.
Línea 8	<code>public CargaLazyUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	<code>Console.WriteLine("=== LAZY LOADING (DEMOSTRACIÓN) ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	<code>var ordenes = _unidad.Ordenes.ObtenerTodasSinInclude();</code> → Obtiene las órdenes neutralizando AutoInclude; así el acceso posterior a Planchas puede demostrar realmente Lazy Loading.
Línea 14	<code>var totalPlanchas = 0;</code> → Inicializa el acumulador que permitirá comprobar cuántas planchas se cargan al acceder a las navegaciones Lazy.
Línea 15	<code>foreach (var orden in ordenes)</code> → Recorre los resultados para evaluar el comportamiento de cada entidad o relación.
Línea 16	<code>totalPlanchas += orden.Planchas.Count;</code> → Accede a la navegación virtual Planchas de cada orden y acumula su tamaño; en 3.9 este acceso es precisamente el que puede disparar una consulta Lazy por entidad.
Línea 17	<code>if (ordenes.Count != 5 totalPlanchas != 5) throw new InvalidOperationException("Lazy Loading no cargó las relaciones.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	<code>Console.WriteLine(\$"Órdenes: {ordenes.Count} Planchas accedidas bajo demanda: {totalPlanchas}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 19	<code>Console.WriteLine("Advertencia docente: el acceso dentro de un bucle puede producir N+1 consultas.");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 20	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 21	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaLazyUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.9 OK");

```

Explicación línea a línea de Program.cs 3.9

Línea 1	using AceriaData.Application.UseCases; → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	using AceriaData.ConsoleApp; → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	using AceriaData.Infrastructure; → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	using Microsoft.Extensions.Configuration; → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	var configuration = new ConfigurationBuilder() → Inicia la construcción de la configuración de la aplicación.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer <code>ConnectionStrings:AceriaDB</code> .
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Obtiene la cadena <code>AceriaDB</code> desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión <code>AceriaDB</code> , evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del <code>composition root</code> .
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra <code>DbContext</code> , repositorio y unidad de trabajo de <code>Infrastructure</code> .
Línea 20	<code>services.AddScoped<CargaLazyUseCase>();</code> → Registra el caso de uso con ciclo de vida <code>Scoped</code> para compartir el mismo ámbito que <code>DbContext</code> .
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios <code>IDisposable</code> al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio <code>Scoped</code> consumido incorrectamente desde un <code>Singleton</code> .
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito <code>Scoped</code> que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.9 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

3.9 OK

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.9, los proxies y las navegaciones `virtual` existen solo para demostrar Lazy Loading y el riesgo N+1. El siguiente checkpoint vuelve a desactivar la configuración de proxies.

Paso 9: Diagnosticar un error común

Error: Acceder a navegaciones Lazy dentro de un bucle puede producir N+1 consultas y depender de un DbContext aún vivo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.9

Diagnóstico técnico

Lazy Loading necesita Proxies, `UseLazyLoadingProxies()` y navegaciones virtuales, además de un DbContext vivo. Como 3.8 introdujo `AutoInclude`, `ObtenerTodasSinInclude()` usa `IgnoreAutoIncludes()` para que la navegación se cargue realmente bajo demanda.

Reto resuelto y verificación adicional

Recorre las cinco órdenes y accede a `orden.Planchas.Count`; el total esperado es cinco. Contrasta este acceso con Eager Loading y explica por qué el patrón puede producir N+1 al crecer N.

Errores comunes revisados

Error	Causa	Solución
Proxy Lazy no intercepta una navegación	La navegación que se quiere cargar bajo demanda no puede ser sobrescrita por el proxy	En el modelo basado en proxies, declarar <code>virtual</code> las navegaciones destinadas a Lazy Loading
Carga Lazy no funciona	No se configuró <code>UseLazyLoadingProxies</code>	Configurar en <code>OnConfiguring</code>
N+1 en bucle	Con Lazy Loading se accede repetidamente a navegaciones todavía no cargadas	Planificar Eager Loading, proyección o Explicit Loading según el caso
<code>ObjectDisposedException</code>	Se accede a propiedades de navegación fuera del ámbito	Usar <code>Include</code> o DTOs

Error	Causa	Solución
Referencia circular en serialización	Las entidades se referencian mutuamente	Usar DTOs o configurar el serializador
NullReferenceException	La propiedad de navegación es null	Comprobar antes de acceder
Proxies no generados	Las clases son sealed	Quitar sealed

Analogía operativa

La carga Lazy en una acería es como pedirle al archivo central que no te traiga las carpetas relacionadas hasta que las pidas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Si después necesitas las planchas, el archivo central te las trae en ese momento. Si necesitas el detalle, te lo trae después. Cada petición adicional es un viaje al archivo central. Si tienes cien órdenes y pides las planchas de cada una, haces cien viajes. Eso es el problema N+1. La carga Eager es como pedir todas las carpetas relacionadas de una vez: un solo viaje con todo lo necesario. La carga Lazy es cómoda porque no tienes que decidir de antemano qué necesitas, pero es ineficiente si haces muchas peticiones. En una acería con mucho volumen, la carga Eager es la opción correcta. En un prototipo o en una aplicación pequeña, la carga Lazy puede ser aceptable. Así funciona la carga Lazy en EF Core: cómoda pero peligrosa, útil en prototipos pero ineficiente en producción.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Cuenta cuántas navegaciones se acceden en el bucle y razona cuántas consultas puede provocar Lazy Loading.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.9 OK, el siguiente estado parte exactamente de esta solución y añade **Explicit Loading**.

Punto 3.10 — Explicit Loading

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.9. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con EnsureCreated().

Objetivo práctico

Implementar y ejecutar **Explicit Loading** sobre AceriaData, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.10
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida M2_2_12_Architecture. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerConCargaExplicita`, `ObtenerConPlanchasPesadasExplicitas`, `IgnoreAutoIncludes`, `IsLoaded`, `Query().Where`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaExplicitaUseCase.cs`

```
using AceriaData.Application.Interfaces;
namespace AceriaData.Application.UseCases;
public sealed class CargaExplicitaUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaExplicitaUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== EXPLICIT LOADING ===");
        var orden = _unidad.Ordenes.ObtenerConCargaExplicita("OF-2024-0001");
        var filtrada = _unidad.Ordenes.ObtenerConPlanchasPesadasExplicitas("OF-2024-0002", 300m);
        if (orden is null || orden.Planchas.Count != 2 || orden.Detalle is null) throw new
        InvalidOperationException("Carga explícita incompleta.");
        if (filtrada is null || filtrada.Planchas.Count != 0) throw new InvalidOperationException("Query() de carga
        explícita inesperado.");
        Console.WriteLine($"Carga completa: {orden.Planchas.Count} planchas | Filtrada OF-0002:
        {filtrada.Planchas.Count}");
    }
}
```

Explicación línea a línea del caso de uso 3.10

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de <code>Application</code> , en especial <code>IUnidadDeTrabajo</code> e <code>IOrdenRepositorio</code> , que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class CargaExplicitaUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de <code>DbContext</code> .
Línea 8	<code>public CargaExplicitaUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	<code>Console.WriteLine("=== EXPLICIT LOADING ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	<code>var orden = _unidad.Ordenes.ObtenerConCargaExplicita("OF-2024-0001");</code> → Carga la orden indicada y, desde <code>Infrastructure</code> , carga explícitamente las navegaciones requeridas antes de devolverla.
Línea 14	<code>var filtrada = _unidad.Ordenes.ObtenerConPlanchasPesadasExplicitas("OF-2024-0002", 300m);</code> → Usa <code>Query()</code> sobre la colección para cargar explícitamente solo las planchas que superan el umbral de peso.

Línea 15	<code>if (orden is null orden.Planchas.Count != 2 orden.Detalle is null) throw new InvalidOperationException("Carga explícita incompleta.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 16	<code>if (filtrada is null filtrada.Planchas.Count != 0) throw new InvalidOperationException("Query() de carga explícita inesperado.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 17	<code>Console.WriteLine(\$"Carga completa: {orden.Planchas.Count} planchas Filtrada 0F-0002: {filtrada.Planchas.Count}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 18	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 19	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaExplicitaUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitaUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.10 OK");
```

CSHARP

Explicación línea a línea de Program.cs 3.10

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa <code>AceriaDbContext</code> para preparar la base de demostración desde el <code>composition root</code> .
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por <code>Migrate</code> , <code>ChangeTracker</code> y las operaciones de consulta del <code>checkpoint</code> .
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa <code>ConfigurationBuilder</code> y las extensiones necesarias para cargar <code>appsettings.json</code> y variables de entorno.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa <code>ServiceCollection</code> , <code>AddScoped</code> , <code>CreateScope</code> y <code>GetRequiredService</code> para componer el grafo de dependencias.
Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Inicia la construcción de la configuración de la aplicación.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Fija como base el directorio de ejecución para localizar <code>appsettings.json</code> .
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer <code>ConnectionStrings:AceriaDB</code> .
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Obtiene la cadena <code>AceriaDB</code> desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión <code>AceriaDB</code> , evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del <code>composition root</code> .
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra <code>DbContext</code> , repositorio y unidad de trabajo de <code>Infrastructure</code> .
Línea 20	<code>services.AddScoped<CargaExplicitaUseCase>();</code> → Registra el caso de uso con ciclo de vida <code>Scoped</code> para compartir el mismo ámbito que <code>DbContext</code> .
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios <code>IDisposable</code> al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio <code>Scoped</code> consumido incorrectamente desde un <code>Singleton</code> .
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito <code>Scoped</code> que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.

Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitaUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.10 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.10 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Cargar explícitamente una navegación que ya estaba cargada puede repetir trabajo; `IsLoaded` permite evitarlo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.10

Diagnóstico técnico

Explicit Loading parte de una entidad rastreada y decide cuándo ejecutar `Collection().Load()` o `Reference().Load()`. `IsLoaded` evita repetir una carga; `Query()` permite filtrar antes de `Load()`. Las consultas raíz neutralizan `AutoInclude` para no falsear la demostración.

Reto resuelto y verificación adicional

Carga 0F-2024-0001: espera dos planchas y detalle. Después carga las planchas de 0F-2024-0002 con peso mínimo 300 kg: espera colección vacía. Así se demuestra que `Query()` altera la consulta de navegación antes de materializar.

Errores comunes revisados

Error	Causa	Solución
Estado de carga no entendido	Se desconoce si la navegación está completa	Consultar <code>IsLoaded</code> para explicitar el estado; si ya es <code>true</code> , <code>Load()</code> es un no-op
Filtro no aplicado	Se aplicó el filtro fuera del Query	Aplicar el filtro dentro del Query
NullReferenceException	La entidad principal es null	Comprobar antes de llamar a <code>Load</code>
Referencia circular	Se carga una entidad que referencia a la principal	Usar DTOs o evitar cargar la referencia inversa
Muchas consultas explícitas	Se llama a <code>Load()</code> para cada entidad dentro de un bucle	Si todas las relaciones se necesitan, valorar Eager Loading o una proyección; si la carga es condicional, medir el coste
Estrategia de carga difícil de razonar	Se mezclan Lazy y Explicit sin intención documentada	Definir qué navegación usa cada estrategia y evitar cargas implícitas inesperadas

Analogía operativa

La carga Explicit en una acería es como pedirle al archivo central que traiga una carpeta relacionada solo cuando se decide que hace falta. `IsLoaded` es como comprobar si ya se dispone de esa carpeta antes de pedirla de nuevo y `Query()` permite pedir solo los documentos que cumplen una condición. Si las relaciones necesarias se conocen desde el principio, Eager Loading o una proyección pueden reducir viajes al archivo; si la necesidad aparece de forma condicional o se quiere filtrar una navegación concreta, Explicit Loading puede resultar más apropiada. No existe una opción universalmente más eficiente: se comparan número de consultas, volumen transferido y claridad del flujo.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Carga únicamente planchas por encima de un peso mínimo mediante `Collection(...).Query()`.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.10 OK, el siguiente estado parte exactamente de esta solución y añade **Composición de consultas y ejecución diferida**.

Punto 3.11 — Composición de consultas y ejecución diferida

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.10. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Composición de consultas y ejecución diferida** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.11
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `BuscarOrdenes`, `Skip`, `Take`, `ToQueryString`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ComposicionConsultasUseCase.cs`

```
using AceriaData.Application.Interfaces;
namespace AceriaData.Application.UseCases;
public sealed class ComposicionConsultasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");
        var resultado = _unidad.Ordenes.BuscarOrdenes("Constructora del Norte", "Pendiente", new DateTime(2024,1,1),
"fecha", true, 1, 10);
        if (resultado.Elementos.Count != 2) throw new InvalidOperationException("Consulta compuesta inesperada.");
        Console.WriteLine($"Resultados: {resultado.Elementos.Count}");
        Console.WriteLine(resultado.Sql);
    }
}
```

CSHARP

Explicación línea a línea del caso de uso 3.11

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application, en especial <code>IUnidadDeTrabajo</code> e <code>IOrdenRepositorio</code> , que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class ComposicionConsultasUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de <code>DbContext</code> .
Línea 8	<code>public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12	<code>Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 13	<code>var resultado = _unidad.Ordenes.BuscarOrdenes("Constructora del Norte", "Pendiente", new DateTime(2024,1,1), "fecha", true, 1, 10);</code> → Compone filtros opcionales, ordenación y paginación y materializa la consulta una sola vez al final.
Línea 14	<code>if (resultado.Elementos.Count != 2) throw new InvalidOperationException("Consulta compuesta inesperada.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 15	<code>Console.WriteLine(\$"Resultados: {resultado.Elementos.Count}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 16	<code>Console.WriteLine(resultado.Sql);</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 17	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 18	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ComposicionConsultasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.11 OK");
```

Explicación línea a línea de Program.cs 3.11

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión <code>AddAceriaInfrastructure</code> , que registra <code>DbContext</code> , repositorio y unidad de trabajo.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa <code>AceriaDbContext</code> para preparar la base de demostración desde el <code>composition root</code> .
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por <code>Migrate</code> , <code>ChangeTracker</code> y las operaciones de consulta del <code>checkpoint</code> .
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa <code>ConfigurationBuilder</code> y las extensiones necesarias para cargar <code>appsettings.json</code> y variables de entorno.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa <code>ServiceCollection</code> , <code>AddScoped</code> , <code>CreateScope</code> y <code>GetRequiredService</code> para componer el grafo de dependencias.
Línea 9	<code>var configuration = new ConfigurationBuilder();</code> → Inicia la construcción de la configuración de la aplicación.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Fija como base el directorio de ejecución para localizar <code>appsettings.json</code> .
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer <code>ConnectionStrings:AceriaDB</code> .
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Obtiene la cadena <code>AceriaDB</code> desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión <code>AceriaDB</code> , evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del <code>composition root</code> .
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra <code>DbContext</code> , repositorio y unidad de trabajo de <code>Infrastructure</code> .
Línea 20	<code>services.AddScoped<ComposicionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida <code>Scoped</code> para compartir el mismo ámbito que <code>DbContext</code> .
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con <code>using</code> para liberar correctamente los servicios <code>IDisposable</code> al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio <code>Scoped</code> consumido incorrectamente desde un <code>Singleton</code> .
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.
Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito <code>Scoped</code> que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.

Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el DbContext para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.11 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.11 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Skip/Take sin un orden determinista produce páginas inestables; la paginación debe seguir a OrderBy.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.11

Diagnóstico técnico

La composición flexible permanece dentro de Infrastructure. Filtros opcionales, orden dinámico y paginación se encadenan sin materializar. `Skip/Take` debe aplicarse tras un orden determinista para que las páginas sean repetibles.

Reto resuelto y verificación adicional

Ejecuta `BuscarOrdenes` con Norte + Pendiente, fecha descendente, página 1 y tamaño 2. El DTO devuelve resultados y `ToQueryString()` de la misma consulta; los filtros ausentes no deben aparecer en SQL.

Errores comunes revisados

Error	Causa	Solución
Materialización prematura	Se llamó a <code>ToList</code> antes de aplicar todos los filtros	Materializar solo al final
Filtros en memoria	Se materializó la consulta antes de filtrar	Construir la consulta con <code>IQueryable</code>
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
<code>ThenBy</code> sin <code>OrderBy</code>	Se llamó a <code>ThenBy</code> sin <code>OrderBy</code> previo	Llamar a <code>OrderBy</code> primero
<code>Skip</code> sin <code>OrderBy</code>	Se aplicó <code>Skip</code> sin ordenar	Aplicar <code>OrderBy</code> antes de <code>Skip</code>
Paginación incorrecta	Se calculó mal el <code>Skip</code>	Usar $(\text{pagina} - 1) * \text{tamanoPagina}$

Analogía operativa

La composición de consultas en una acería es como construir una orden de búsqueda al archivo central paso a paso. En lugar de pedir todos los documentos y después filtrarlos, el jefe de planta va añadiendo condiciones a la orden: primero el cliente, después el estado, después el rango de fechas. Cada condición se añade a la orden sin enviarla todavía. Solo cuando la orden está completa, el jefe la envía al archivo central. El archivo recibe la orden con todas las condiciones y devuelve solo los documentos que cumplen todas ellas. La ejecución diferida es como preparar la orden sin enviarla: se puede modificar hasta el último momento. La materialización prematura es como enviar la orden antes de terminarla: el archivo devuelve documentos que después hay que filtrar a mano. La composición de consultas permite construir consultas flexibles y eficientes, adaptadas a las necesidades de cada momento.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Añade un filtro opcional y una segunda forma de ordenación sin ejecutar la consulta antes de `Skip/Take`.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.11 OK, el siguiente estado parte exactamente de esta solución y añade **Buenas prácticas en el acceso a datos y composición de consultas**.

Punto 3.12 — Buenas prácticas en el acceso a datos y composición de consultas

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.11. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Buenas prácticas en el acceso a datos y composición de consultas** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.12
dotnet restore AceriaData.sln
```

POWERSHELL

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos añadidos en este estado son: `ObtenerResumenesPendientesOptimizado`, `ExisteAlgunaOrdenPendiente`, `ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano` y `ObtenerPorNumeroOptimizado`. Como cierre de arquitectura se retiran `Consulta` y `ObtenerSqlFundamentos` del puerto público; al retirar `AutoInclude` como comportamiento global también desaparecen `ObtenerOrdenesAutoInclude` y `ObtenerOrdenesIgnorandoAutoInclude`. La composición queda encapsulada en `Infrastructure`. El `AsSplitQuery()` final se conserva sobre dos colecciones hermanas —Planchas y OrdenesAleaciones— más Detalle.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/BuenasPracticasUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class BuenasPracticasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");
        var resmenes = _unidad.Ordenes.ObtenerResumenesPendientesOptimizado();
        var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();
        var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();
        var inexistente = _unidad.Ordenes.ObtenerPorNumeroOptimizado("0F-2024-9999");
        if (resmenes.Count != 3 || !existe || split.Count != 5 || inexistente is not null) throw new
InvalidOperationException("Buenas prácticas: validación E2E fallida.");
        Console.WriteLine($"Pendientes proyectadas: {resmenes.Count} | Any: {existe} | SplitQuery: {split.Count}");
        Console.WriteLine("El contrato final ya no expone IQueryable fuera de Infrastructure.");
    }
}
```

CSHARP

Explicación línea a línea del caso de uso 3.12

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa los puertos de Application, en especial <code>IUnidadDeTrabajo</code> e <code>IOrdenRepositorio</code> , que desacoplan el caso de uso de EF Core.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Declara el espacio de nombres de la capa a la que pertenece el archivo.
Línea 5	<code>public sealed class BuenasPracticasUseCase</code> → Declara el caso de uso concreto que coordina la demostración del punto.
Línea 6	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de <code>DbContext</code> .
Línea 8	<code>public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Recibe la unidad de trabajo mediante inyección de dependencias.
Línea 10	<code>public void Ejecutar()</code> → Define la operación docente que ejecutará el composition root.
Línea 11	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 12	<code>Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13	<code>var resmenes = _unidad.Ordenes.ObtenerResmenesPendientesOptimizado();</code> → Ejecuta una lectura proyectada y no-tracking de las órdenes pendientes, sin exponer IQueryable al caso de uso.
Línea 14	<code>var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();</code> → Ejecuta Any para expresar directamente la comprobación de existencia de órdenes pendientes.
Línea 15	<code>var split =</code> <code>_unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();</code> → Carga el grafo final con AsSplitQuery sobre dos colecciones hermanas y una referencia para controlar la multiplicación de filas.
Línea 16	<code>var inexistente = _unidad.Ordenes.ObtenerPorNumeroOptimizado("0F-2024-9999");</code> → Ejecuta una búsqueda de solo lectura con FirstOrDefault y valida explícitamente el caso sin resultado.
Línea 17	<code>if (resmenes.Count != 3 !existe split.Count != 5 inexistente is not null) throw</code> <code>new InvalidOperationException("Buenas prácticas: validación E2E fallida.");</code> → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.
Línea 18	<code>Console.WriteLine(\$"Pendientes proyectadas: {resmenes.Count} Any: {existe} SplitQuery:</code> <code>{split.Count}");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 19	<code>Console.WriteLine("El contrato final ya no expone IQueryable fuera de</code> <code>Infraestructure.");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.
Línea 20	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.
Línea 21	<code>}</code> → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB");
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.12 OK");
```

Explicación línea a línea de Program.cs 3.12

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa AceriaDbContext para preparar la base de demostración desde el composition root.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.
Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Inicia la construcción de la configuración de la aplicación.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Fija como base el directorio de ejecución para localizar appsettings.json.
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Carga la configuración JSON obligatoria que contiene la cadena de conexión.
Línea 12	<code>.AddEnvironmentVariables()</code> → Permite sobrescribir configuración mediante variables de entorno.
Línea 13	<code>.Build();</code> → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Obtiene la cadena AceriaDB desde la configuración.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.
Línea 18	<code>var services = new ServiceCollection();</code> → Crea el contenedor de servicios del composition root.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.
Línea 22	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.
Línea 23	<code>{</code> → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.
Línea 24	<code>ValidateOnBuild = true,</code> → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.
Línea 25	<code>ValidateScopes = true</code> → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.
Línea 26	<code>});</code> → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27	<code>using var scope = provider.CreateScope();</code> → Crea el ámbito Scoped que compartirá <code>AceriaDbContext</code> , <code>IUnidadDeTrabajo</code> y el caso de uso durante esta ejecución.
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> para preparar la base de demostración.
Línea 30	<code>context.Database.EnsureDeleted();</code> → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.
Línea 31	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.
Línea 32	<code>DemoData.Seed(context);</code> → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.
Línea 33	<code>context.ChangeTracker.Clear();</code> → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.
Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();</code> → Resuelve el caso de uso concreto registrado para este checkpoint.
Línea 36	<code>useCase.Ejecutar();</code> → Ejecuta la demostración y sus aserciones E2E.
Línea 38	<code>Console.WriteLine("3.12 OK");</code> → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La compilación debe terminar sin errores. `Application` no referencia `Entity Framework Core`; todas las llamadas específicas de EF permanecen en `Infrastructure` o en el `composition root`.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el `Change Tracker` y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.12 OK
```

TEXT

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.12, `IOrdenRepositorio` ya no devuelve `IQueryable<OrdenFabricacion>`; el objetivo es cerrar el módulo con una frontera limpia entre `Application` e `Infrastructure`.

Paso 9: Diagnosticar un error común

Error: Exponer `IQueryable` desde `Application` deja que capas superiores acoplen su lógica al proveedor; el contrato final lo elimina.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.12

Diagnóstico técnico

El cierre conserva `AsNoTracking`, proyecciones, `Any`, `FirstOrDefault` y `SplitQuery` cuando el shape lo justifica. Retira `IQueryable` del puerto y `AutoInclude` del modelo para evitar decisiones implícitas en Application.

Reto resuelto y verificación adicional

Valida tres pendientes proyectadas, `Any == true`, cinco órdenes en la consulta `SplitQuery` y `null` para `0F-2024-9999`. El `SplitQuery` final carga Planchas, OrdenesAleaciones→Aleacion y Detalle.

Errores comunes revisados

Error	Causa	Solución
AsNoTracking en consulta de escritura	Se aplicó AsNoTracking a una consulta que modifica datos	Usar AsNoTracking solo en consultas de solo lectura
Count() > 0 en lugar de Any	Se usó Count para saber si hay elementos	Usar Any
First en lugar de FirstOrDefault	Se usó First cuando puede no haber resultados	Usar FirstOrDefault
Multiplicación de filas	Se cargaron colecciones hermanas al mismo nivel en modo single-query	Comparar <code>AsSingleQuery()</code> y <code>AsSplitQuery()</code> para el volumen real
Materialización prematura	Se llamó a ToList antes de aplicar todos los filtros	Materializar solo al final
N+1	Se accede en un bucle a navegaciones Lazy todavía no cargadas	Elegir de forma explícita Eager Loading, proyección o Explicit Loading
Falta de documentación	No se documentaron las decisiones	Añadir comentarios XML

Analogía operativa

Las buenas prácticas de acceso a datos en una acería son como las normas de seguridad y eficiencia de la planta. No se trata solo de producir acero, sino de producirlo de forma segura, eficiente y sostenible. Usar proyecciones es como pedir solo los datos que se necesitan: no se pide la carpeta completa si solo se necesita el número de orden. Usar `AsNoTracking` es como no registrar cada plancha en el libro de producción si solo se va a consultar. Usar `Any` es como comprobar si hay planchas en el almacén sin contarlas todas. Usar `FirstOrDefault` es como buscar una plancha por su número y devolver `null` si no existe, en lugar de lanzar una alarma. Usar `AsSplitQuery` es como dividir una orden de búsqueda grande en varias más pequeñas para evitar un envío masivo. Documentar las decisiones es como escribir las normas en el manual de la planta para que todos las conozcan. Así funcionan las buenas prácticas en EF Core: producen código más eficiente, más mantenible y más predecible.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Revisa `IOrdenRepositorio` y justifica por qué ya no contiene `IQueryable` aunque Infrastructure siga componiendo consultas.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar `3.12 OK`, el siguiente estado parte exactamente de esta solución; con ello queda cerrado el Módulo 3.